

RDHIS2 Mobile

DHIS2 Web API Reference

Every endpoint the application calls · method rules · exact request shape per call site

HISP Ethiopia — Mobile team

Document version 1.0 · 10 September 2026 · verified against the source at the current head

Companion to docs/19-api-reference.md (source of truth) and the RDHIS2 Mobile Technical Documentation.

1 Conventions

If an endpoint is not in this document, the app does not call it.

Aspect	Value
Base URL	{server}/api — compiled default <code>https://hmis-staging.moh.gov.et/api</code> (<code>ApiConstants.baseUrl</code>). Runtime-overridable from the login screen or Settings; normalised to <code>https://+ /api</code> .
Authentication	HTTP Basic on every request — <code>Authorization: Basic base64(username:password)</code> . Built in one place (<code>AuthInterceptor.buildBasicToken</code>). No tokens, no OAuth.
Default headers	<code>Content-Type: application/json</code> , <code>Accept: application/json</code> . The connectivity probe additionally sends <code>X-Requested-With: XMLHttpRequest</code> .
Timeouts	30 s each for connect / receive / send.
Response decoding	Large JSON bodies decoded on a background isolate (<code>Dio BackgroundTransformer</code>) so the UI never freezes during a metadata sync.
Paging	Every metadata list request sends <code>paging=false</code> — whole (scoped) collections, never pages.
Field selection	Every request sends a <code>DHIS2 fields=</code> selector; the app never fetches *.
Server-side filtering	<code>filter=</code> clauses; more than one is OR-combined with <code>rootJunction=OR</code> .

1.1 The two HTTP clients

Client	Interceptors	Used for
<code>ApiClient()</code> (singleton)	<code>AuthInterceptor</code> → <code>LoggingInterceptor</code> . A 401 anywhere ends the session and redirects to <code>/login?reason=session-expired</code> .	Metadata sync, form value pull/push, capture browse, dashboards / analytics, audits, icons.
<code>ApiClient.withBasicAuth(...)</code>	None — a fixed <code>Authorization</code> header, no plugins. A 401 here never ends the session.	The login credential probe, the per-session sync client on <code>AppSession.instance.api</code> , the connectivity probe, and tests.

2 Method Rules

The app is read-mostly. It issues GET for everything except three write calls.

Method	Where the app uses it	Rules
GET	All metadata, all reads, login, ping, analytics, audits, icons.	Never mutates server state. Every GET at a feature boundary is best-effort: on failure the caller falls back to the local database or a cache and the UI is never blocked. Metadata GETs always carry <code>fields= + paging=false</code> .
POST	<code>/api/dataValueSets.json</code> (bulk data value import) and <code>/api/completeDataSetRegistrations</code> (mark a form complete).	Both idempotent by composite key — re-sending the same values / registration is safe, which is what makes the offline retry loop correct. Data import uses <code>importStrategy=CREATE_AND_UPDATE + atomicMode=NONE</code> (see §5).
DELETE	<code>/api/completeDataSetRegistrations</code> (re-open a completed form).	Addressed by query params (<code>ds</code> , <code>pe</code> , <code>ou</code>), plus <code>cc</code> / <code>cp</code> for a non-default attribute option combo — the default combo sends neither. A 409 here is treated as permanent (row marked error, not retried).
PUT / PATCH	Never. The app issues no PUT or PATCH request anywhere.	Updates to an existing data value go through the same POST <code>/api/dataValueSets.json</code> — DHIS2's <code>dataValueSets</code> import is an upsert, so a separate update verb is unnecessary and would complicate the single-push-path offline model.

The app never creates or edits server-side metadata, users, visualizations, dashboards, option sets, or organisation units. Metadata “deletes” during a delta sync are local database deletes only, never an API call.

3 Endpoint Catalogue

3.1 Authentication & session

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/me.json	Validate credentials on online login; capture the Date response header to anchor the tamper-resistant clock.	fields=id	SessionService._serverAccepts
GET	/api/me.json	Cache the user row for offline login and read the capture-root org units (id, level) that scope the org-unit sync.	fields=id,username,displayName,organisationUnits[id,level]	MetadataSyncService._fetchUserCaptureRoots
GET	/me	Transitional legacy glue: after an online login, cache the Basic token + full /me payload some older SecureStorage-reading code still expects.	fields=id,username,firstName,surname,email,phoneNumber,avatar,authorities,organisationUnits[id,displayName,shortName,code,level,path]; validateStatus < 500	AuthRemoteDataSourceImpl.login
GET	/api/system/ping	App-wide connectivity ground truth — probed every 30 s, on every OS connectivity event, and on checkNow().	none (bare client)	ConnectivityService.checkNow

Rules: 401 on the first row → invalidCredentials. Any other error on the first two rows → the caller falls back to the offline verifier path / retries next login. For the ping, any HTTP response (incl. 401) counts as online; only a transport error is offline.

3.2 Metadata synchronization

All metadata GETs go through MetadataResource and share three shapes:

Method	Endpoint	Purpose	Parameters
GET	/api/{resource}.json	Full sync — every object of a resource.	fields=<selector>, paging=false [, filter=<clauses>, rootJunction=OR when >1]
GET	/api/{resource}.json	Delta sync, step 1 — cheap change list.	fields=id,lastUpdated, paging=false [, filter...]
GET	/api/{resource}.json	Delta sync, step 2 — only changed/new objects, chunked at 100 ids/request.	fields=<selector>, filter=id:in:[...], paging=false

Resources, in dependency order, with the exact fields selector

#	resource	fields selector	Notes
1	categoryOptions	id,name,displayName,shortName,startDate,endDate,lastUpdated	
2	categories	id,name,displayName,dataDimension	

		Type, categoryOptions[id], lastUpdated	
3	categoryCombos	id, name, displayName, dataDimensionType, skipTotal, categories[id], lastUpdated	
4	categoryOptionCombos	id, name, categoryCombo[id], categoryOptions[id], lastUpdated	
5	optionSets	id, name, displayName, valueType, lastUpdated, attributeValues[value, attribute[id]]	
6	options	id, code, name, displayName, sortOrder, optionSet[id], lastUpdated	
7	attributes	id, name, displayName, valueType	No lastUpdated → delta falls back to a full syncAll.
8	dataElements	id, name, displayName, formName, description, valueType, categoryCombo[id], optionSet[id], lastUpdated, attributeValues[value, attribute[id]]	
9	indicators	id, name, displayName, numerator, denominator, description, annualized, indicatorType[factor], lastUpdated, attributeValues[...]	
10	dataElementGroups	id, name, displayName, dataElements[id], lastUpdated, attributeValues[...]	
11	organisationUnits	id, name, displayName, parent[id, name], path, code, openingDate, closedDate, lastUpdated	Scoped: filter=path:like:<captureRootUid> per root, OR-combined. Depth bounded to root + direct children client-side after the fetch (OrgUnitResource.isValid).
12	dataSets	id, name, displayName, periodType, version, openFuturePeriods, expiryDays, lastUpdated, categoryCombo[id], attributeValues[...], dataSetElements[sortOrder, compulsory, categoryCombo[id], dataElement[id, categoryCombo[id]]], compulsoryDataElementOperations[dataElement[id], categoryOptionCombo[id]], organisationUnits[id]	
13	sections	id, name, displayName, sortOrder, lastUpdated, dataSet[id], dataElements[id], indicators[id], greyedFields[dataElement[id], categoryOptionCombo[id]]	
14	validationRules	id, name, displayName, description, importance, operator, instruction, periodType, lastUpdated, leftSide[expression, description, missingValueStrategy], rightSide[...]	

One targeted metadata GET outside the resource loop:

Method	Endpoint	Purpose	Parameters	Caller
GET	/api/categoryCombos.json	Resolve the canonical default category combo + its unique COC on an instance that has duplicated the default COC.	filter=name:eq:default, fields=id,name,displayName,categoryOptionCombos[id,name]	fetchCanonicalDefaultCombo

3.3 Data capture — value pull & push

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/dataValueSets.json	On form open (online): pull the server's current values for one dataset/period/org unit, for per-cell conflict resolution.	dataset, period, orgUnit	DataValueSync._pullAndResolve
GET	/api/dataValueSets.json	Build the outlier-detection history snapshot — ~25 months of this dataset's values at this org unit.	dataset, orgUnit, startDate, endDate (~770 days)	OutlierDetectionService.fetchHistory
POST	/api/dataValueSets.json	Push queued (pending) data values. Bulk import, ≤ 500 values/request, chunked recursively.	body {dataValues: [{dataElement, period, orgUnit, categoryOptionCombo, attributeOptionCombo, value, comment?, deleted?}]}; query importStrategy=CREATE_AND_UPDATE, atomicMode=NONE	pushDataValueBatch

On failure: the pull returns null (local data stands); the outlier fetch falls back to the cached snapshot; the push — on a transport failure — leaves every value pending for the next retry (a 409 is a verdict, not a failure — see §5).

Payload entry rules (_payloadEntry)

- `value` is always trimmed before sending.
- A cleared cell (no value and no comment) is sent as ``deleted": true` — not ``value": ""`, which DHIS2 rejects with E7610. A deleted entry removes the server value if present and is silently ignored if it never existed.
- `comment` is included only when non-null.
- Routine datasets use the dataset instance's default attributeOptionCombo; Disease Registration datasets thread the picked combo through.

3.4 Completion registrations

Method	Endpoint	Purpose	Key parameters	Caller
POST	/api/completeDataSetRegistrations	Mark a form complete. One call per pending registration.	body {completeDataSetRegistrations:	CompletenessSync.pushPending

			[{dataSet,period,organisationUnit,attributeOptionCombo}]]}	
DELETE	/api/completeDataSetRegistrations	Re-open a completed form (un-complete).	ds, pe, ou [, cc, cp for a non-default attributeOptionCombo — the default combo sends neither]	CompletenessSync.pushPending (completed==false)
GET	/api/completeDataSetRegistrations.json	Pull the server's completion state so a report finished on the web drops out of the "To-do" list. Org units fanned out ≤ 40/request.	orgUnit (list), startDate, endDate, fields=dataset,period,organisationUnit,attributeOptionCombo,completed,date	CompletenessSync.pullRecent

On failure: POST/DELETE — a 409 marks the row error with the server's reason (permanent, not retried); any other DioException leaves it pending. GET pullRecent is best-effort; the local view is left as-is.

3.5 Audit trail

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/audits/dataValue.json	DHIS2's own server-side audit trail for one cell — the "Show history" query.	de, pe, ou [, cc, cp], pageSize (default 50)	ServerAuditService.fetchForCell

On failure: returns null; the history sheet shows only the local AuditLogStore trail. Server-side auditing must be enabled for any rows to exist — an empty result does not prove nothing changed.

3.6 Organisation-unit browse (Capture)

These supplement, never replace, the locally-synced depth-bounded tree.

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/organisationUnits.json	Live children of one node (browse below the offline depth bound). Not persisted.	filter=parent.id:eq:<id>, fields=id,displayName,path,children[id],paging=false	CaptureRepositoryImpl._fetchChildrenLive
GET	/api/organisationUnits.json	Batched "which of these have children" check — one request for the expand-arrow decision.	filter=parent.id:in:[...], fields=parent[id], paging=false	CaptureRepositoryImpl._liveChildrenExistence
GET	/api/organisationUnits/{id}.json	On visiting an org unit: fetch it + its assigned dataset ids, mirror the org unit and its dataset links locally (links replaced, not merged).	fields=id,name,displayName,parent[id,name],path,code,openingDate,closedDate,lastUpdated,dataSets[id]	CaptureRepositoryImpl._fetchAndCacheVisitedOrgUnit

3.7 Data-entry form metadata

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/dataSets/{id}.json	Resolve a dataset's categoryCombo +	fields=categoryCombo[id,na	DataEntryRepositoryImpl._fetchDefault

		its COCs when missing from the local cache; persist both.	me,displayName,categoryOptionCombos[id,name]]	tComboFromDataSet
--	--	---	---	-------------------

3.8 Visualization — server dashboards (read-only)

Nothing here is written to the local database; only the last analytics answer is cached (under a SyncInfoTable key) so a chart stays viewable offline.

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/dashboards.json	List server dashboards.	fields=id,name, paging=false	ChartRepositoryImpl.getDashboards
GET	/api/dashboards/{id}.json	The visualization items on one dashboard (MAP / APP items skipped).	fields=dashboardItems[type, visualization[id,name,type]]	ChartRepositoryImpl.getDashboardVisualizations
GET	/api/visualizations.json	Flat reference list of visualizations the user can see.	fields=id,name,type, paging=false	ChartRepositoryImpl.getServerVisualizations
GET	/api/visualizations/{id}.json	One visualization's own definition — its column / row / filter dimensions and items.	fields=id,name,type,columns[dimension,items[id,displayName]],rows[...],filters[...]	ChartRepositoryImpl.runServerVisualization
GET	/api/analytics.json	Run a server visualization's own query. columns+rows → dimension=; filters → filter=.	dimension (repeated), filter (repeated), includeMetadataDetails=false	ChartRepositoryImpl.runServerVisualization

3.9 Visualization — local dashboards & the chart builder

Builder pickers prefer the live server and fall back to locally-synced metadata (or, for org units, the depth-bounded local tree) when offline.

Method	Endpoint	Purpose	Key parameters	Caller
GET	/api/indicatorGroups.json	Indicator-group picker.	fields=id,displayName, paging=false	getIndicatorGroups
GET	/api/indicatorGroups/{id}.json	Indicators within a group (indicator groups aren't synced offline — falls back to a flat local indicator list).	fields=indicators[id,displayName]	getIndicatorsInGroup
GET	/api/dataElementGroups.json	Data-element-group picker.	fields=id,displayName, paging=false	getDataElementGroups
GET	/api/dataElementGroups/{id}.json	Aggregatable data elements of a group, each with its COCs.	fields=dataElements[id,displayName,domainType,categoryCombo[categoryOptionCombos[id,displayName]]]	getDataElementsInGroup
GET	/api/dataSets.json	Dataset picker.	fields=id,displayName, paging=false	getDataSets
GET	/api/organisationUnits.json	Live org-unit children for the builder's picker — deliberately the full hierarchy, never cached.	filter=parent.id:eq:<id>, fields=id,displayName,path,level,children[id],	getOrgUnitChildrenLive

			paging=false	
GET	/api/analytics.json	Run one saved local chart.	dimension=dx:<items>;pe:<period> (repeated), filter=ou:<orgUnit>, includeMetadataDetails=false	LocalVisualizationRepositoryImpl.runChart

Local chart configurations and their cached results are stored on the device only (JSON under savedCharts / chartCache_<id> keys in SyncInfoTable) — no server object is ever created or updated.

3.10 Assets

Method	Endpoint	Purpose	Notes	Caller
GET	/api/icons/{iconKey}/icon.svg	The DHIS2 icon for a dataset card.	responseType: bytes; one fetch per key per run (in-memory cache); a failure just shows the fallback icon.	DataSetIcon._fetch

4 Delta-Sync Algorithm (per resource)

1. GET /api/{resource}.json?fields=id,lastUpdated&paging=false (+ scope filter).
2. Compare each (id, lastUpdated) against the local row (1-second tolerance — Drift stores DateTime at second precision, the server sends milliseconds).
3. GET ...&filter=id:in:[...]&fields=<full selector> for the changed/new ids, chunked at 100 ids per request (long id:in URLs break on proxies).
4. Delete locally any id the server's list no longer contains — a local delete, no API call.
5. Resources with no lastUpdated column (attributes) cannot delta — they fall back to a full syncAll.

A network drop mid-sequence leaves earlier resources fully synced and later ones untouched: stale but internally consistent, never half-written. lastMetadataSync is written only after a full pass completes.

5 Data-Push Verdict Parsing (POST /api/dataValueSets.json)

Server response	Meaning	App action
HTTP 200, no conflicts, no ignored	Whole batch accepted.	Every value in the batch → synced.
HTTP 200 or 409 with an ImportSummary body	Partial: atomicMode=NONE judges each value individually. Each conflict's indexes (or top-level rejectedIndexes) maps back to the exact payload position.	Rejected values → error + the server's message; all others → synced.
HTTP 409 on an older server with no index info	Conflicts present but not located.	Best-effort substring match of the conflict text against each value's dataElementUid / period.
No parseable ImportSummary (true transport failure)	The request never reached the server, or the body is unusable.	Nothing is marked. Every value stays pending for the next retry.

Batching: _maxBatchSize = 500 values per request, chunked recursively — a mid-transfer drop only loses the unsent slice. Completion-registration verdicts follow the same “a 409 is a verdict, a transport error is not” rule, but a 409 there is permanent (retrying an already-rejected registration cannot succeed).

6 Summary — one line per endpoint

Method	Endpoint	Read / Write	Retriable
GET	/api/me.json	read	yes
GET	/me	read	yes
GET	/api/system/ping	read	yes
GET	/api/{resource}.json (14 metadata resources)	read	yes
GET	/api/categoryCombos.json?filter=name:eq:default	read	yes
GET	/api/dataValueSets.json	read	yes
POST	/api/dataValueSets.json	write (upsert)	yes — idempotent by key
POST	/api/completeDataSetRegistrations	write	yes — idempotent by key
DELETE	/api/completeDataSetRegistrations	write	yes — but 409 is permanent
GET	/api/completeDataSetRegistrations.json	read	yes
GET	/api/audits/dataValue.json	read	yes
GET	/api/organisationUnits.json	read	yes
GET	/api/organisationUnits/{id}.json	read	yes
GET	/api/dataSets/{id}.json	read	yes
GET	/api/dataSets.json	read	yes
GET	/api/dashboards.json	read	yes
GET	/api/dashboards/{id}.json	read	yes
GET	/api/visualizations.json	read	yes
GET	/api/visualizations/{id}.json	read	yes
GET	/api/analytics.json	read	yes
GET	/api/indicatorGroups.json	read	yes
GET	/api/indicatorGroups/{id}.json	read	yes
GET	/api/dataElementGroups.json	read	yes
GET	/api/dataElementGroups/{id}.json	read	yes
GET	/api/icons/{iconKey}/icon.svg	read	yes

No PUT. No PATCH. Three write calls, all idempotent, all on the same offline retry queue.